

Kernel Design in SGLang

Deep Dive into SGLang — Chapter 26

Yin Yang

Foundation Books Project

Where this chapter sits

This is the chapter that turns “call a fast kernel” into “prove every layer still names the same operation.”

- Custom kernels exist where generic tensor operators leave hot-path cost exposed: extra launches, extra memory passes, no hardware-specific path.
- Replacing a generic tensor expression with a custom launch is only a **serving feature** if the wrapper, schema, launch, and tests all describe one operator.
- The dangerous failure is **shape-preserving**: a bad launch returns a tensor of the right rank while multiplying the wrong half or mutating the wrong buffer.

Goal: read a kernel and say what it promises, where that promise is checked, and how a test proves it still holds.

The operator-contract idea

Launch, boundary, cost

Specialize, measure, accept

The operator-contract idea

The running example: `silu_and_mul`

One fused activation-and-multiply operator carries the whole chapter.

- Input has shape $(\dots, 2d)$; split the last dimension into gate g and up u .
- The operator computes, with ϕ the selected activation,

$$\text{out} = \phi(g) \odot u, \quad \phi = \text{SiLU for } \text{silu_and_mul}.$$

- Fusing activation and multiply avoids materializing $\phi(g)$ as an intermediate — one fewer memory pass on a hot MLP path.

Same formula at every layer; only the implementation changes. A $(\dots, 2d) \rightarrow (\dots, d)$ shape rule is the spine that must survive from Python to CUDA to test.

The kernel contract, as one map

The primary mental model is compact:

$$\mathcal{K} : (X, M, \theta) \mapsto Y.$$

- X : tensor inputs and mutable buffers.
- M : layout / metadata — shape, dtype, stride, device, alignment, workspace.
- θ : scalar arguments (epsilon, top- k , alignment assumptions).
- Y : output state, including caller-visible mutation.

Most kernel bugs in serving are contract bugs: M or θ silently changes the meaning of X . The expanded tuple names the same pieces finely: shapes, dtype set Δ_{dtype} , layout rules Λ_{layout} , scalar args Θ_{arg} , side effect ω , tolerance ϵ .

The evidence chain: one operator, five layers

Layer	What it carries for <code>silu_and_mul</code>
Python wrapper	input is $(\dots, 2d)$; out is $(\dots, d)$; reject bad shapes, allocate output, check 16-byte alignment.
Extension schema	<code>silu_and_mul(Tensor!out, Tensorinput) -> ()</code> ; <code>Tensor!</code> marks caller-visible mutation ω .
C++/CUDA launch	derive d , flatten to <code>num_tokens</code> , pick stream + device guard, dispatch dtype, launch.
Reference test	<code>silu(x[...,:d])*x[...,:d]</code> ; check the mutated out within ϵ .

A faster launch is evidence for the same operator only when every row above still describes it.

From concept to code: the wrapper is the contract gate

The wrapper makes the $(\dots, 2d) \rightarrow (\dots, d)$ rule executable before any device code runs.

```
from sgl_kernel/python/sgl_kernel/elementwise.py

def silu_and_mul(input, out=None):
    if input.shape[-1] * input.dtype.itemsize % 16 != 0:
        raise ValueError("The pointers must be multiple of 16 bytes.")
    if out is not None:
        _check_shape(input, out)    # equal rank, equal leading dims, 2d -> d
    else:
        out = torch.empty(input.shape[:-1] + (input.shape[-1] // 2,), ...)
    torch.ops.sgl_kernel.silu_and_mul.default(out, input)
    return out
```

Shape gate, alignment gate, output ownership — all fixed here before the call crosses into the registered operator.

Launch, boundary, cost

What the launch adds on top of the contract

The wrapper fixed *what*; the launch fixes *how it runs*.

- **Token count and split width.** $d = \text{input.size}(-1)/2$; `num_tokens` = flattened leading dims. Grid is one block per token row.
- **Vector width from alignment.** `vec_size = 16/sizeof(c_type)` — coalesced, packed loads, legal only because the wrapper checked 16-byte alignment.
- **Stream + device guard.** inherited from the PyTorch execution context, not an implicit default.
- **Dtype dispatch.** unsupported dtypes fail at the dispatch macro, never reach device code with an ambiguous type.

Backend-specific mechanics (HIP on ROCm, FlashInfer on CUDA) preserve one Python-visible result.

The kernel boundary is a correctness boundary

Unsupported shapes, dtypes, and layouts must **fail or fall back before device code mutates outputs**.

Invariant (Kernel-contract alignment)

A custom kernel is valid only when the Python wrapper, extension schema, signature, shape/dtype checks, device and stream ownership, mutation semantics, compile compatibility, reference path, and tolerance all describe the same operator contract.

Shape accounting is only the first layer: dtype accounting picks the legal template, device accounting ties the launch to the caller's stream, and side-effect accounting names every mutated buffer.

Fused mutation kernels: shape survives, meaning may not

The same discipline governs hotter kernels between model algebra and attention:

Kernel	Shape-preserving failure
Gated activation & multiply	halves swapped, or wrong activation variant.
Fused add + RMSNorm	normalized input is fine; residual stream left stale.
Cache store	every tensor keeps its shape; K/V lands in the wrong slot.
Rotary embedding	rows keep shape but receive another token's rotation.
Fused QK-norm + RoPE	reference and fused disagree on which heads/dims were normalized.
Top- k renormalization	probability tensor legal; selected set no longer matches policy.

Each row can pass a rank-and-shape check while corrupting the next operator's input.

Specialize, measure, accept

Specialization buys speed by adding validation surface

- A fast path must **state where it is valid**, **route away** unsupported inputs, **preserve** mutation and numerics, and stay **testable** as backends evolve.
- Maintenance burden = number of visible contract surfaces that must stay aligned: wrappers, schemas, launch code, generated variants, backend branches, oracles, measurement rows.
- Wrapper convenience does not remove lower-level defense: the custom all-reduce C++ path rechecks dtype, count, contiguity, capacity even though Python already did.

A tolerance hidden in prose is not a contract; it must live in an executable assertion.

A kernel claim is only as big as its measured row

The measured `silu_and_mul` row (RTX 5090, `sglang-kernel`, eager launch, non-PDL):

Dtype	Input	Supplied μs	Ref μs	Ref/custom
float16	(1, 8192)	7.30	12.70	1.74
float16	(128, 22016)	7.20	13.22	1.84
bfloat16	(16, 8192)	6.94	12.90	1.86

The row also records negatives: a wrong out width raises `AssertionError` (`8192!=8194`); a non-16-byte input raises `ValueError`. And it is bounded — CUDA graph replay and PDL are a *separate* timing mode.

What to carry forward

1. A kernel is a **contract** $\mathcal{K} : (X, M, \theta) \mapsto Y$, not just a fast function.
2. Correctness is **alignment** across wrapper, schema, launch, backend, oracle — one operator at every layer.
3. Fused mutation kernels fail **shape-preservingly**; tests must check *every* caller-visible write, not just the returned shape.
4. Specialization **narrows the legal surface and widens the review surface**; the tolerance is part of the contract.
5. A speed claim is bounded to its **measured row** — host, package, timing mode, PDL status.

The durable rule: follow the operator, not the file tree.